

گزارش سوال 3 عملی - تمرین ۲ - سیستم عامل

Fork Bomb

۱. یک نوع حمله ساده اما مخرب است که با درگیر کردن تمام منابع سیستم (مانند cpu و ram)، باعث hang یا crash کردن سیستم می‌شود. در این حمله، هر process با دستور fork() یک یا چند process جدید ایجاد کرده که هر یک از آن‌ها مجدداً همین کار را می‌کنند و تعداد فرایندها به طور نمایی افزایش می‌یابد.

۲. مثال ۱:

```
:( ){ :|:& };;
```

توضیح:

: () → تعریف یک تابع به نام : (دو نقطه)

{ :|:& } → بدنه تابع:

: | → تابع خودش را فراخوانی می‌کند و خروجی را به ورودی خودش پایپ می‌کند

& → این کار را در پس‌زمینه (background) انجام می‌دهد

:: → تابع را فراخوانی می‌کند

مثال ۲:

```
bomb() {  
    bomb | bomb &  
}  
bomb
```

تابع bomb خودش را دو بار فراخوانی می‌کند: یکی در پایپ، یکی در پس‌زمینه.

مثال ۳:

```
#include <unistd.h>  
int main() {
```

```

while (1) {
    fork();
}
return 0;
}

```

حلقه بی‌نهایت fork() باعث ایجاد فرآیندهای جدید می‌شود. هر فرآیند جدید هم همین حلقه را اجرا می‌کند.

مثال اول را بر روی vm اجرا کردیم. پس از مدت کوتاهی تعداد نامتناهی ارور مبتنی بر عدم امکان ایجاد process جدید به علت کمبود منابع سیستم پرینت میشد. با Ctrl + C هم نمیشد آن‌ها را متوقف کرد. نهایتاً ترمینال را بستیم. اما دیگر برنامه‌ها نیز اجرا نمی‌شدند و نهایتاً سیستم هنگ کرد و هیچ کاری انجام نمیداد. در آخر vm را متوقف کردیم. اروری که در ترمینال مشاهده میشد:

fork: retry: Resource temporarily unavailable

۳. تحلیل منابع:

حافظه: پس از مدت کوتاهی تمام ram پر میشود. سپس بخش swap دیسک نیز پر شده و در این مرحله I/O خیلی زیادی با دیسک ایجاد می‌شود که سربار زیادی دارد. نهایتاً OOM Killer کرنل فعال شده و با ارور out of memory فرایند‌ها را میکشد.

پردازنده: پس از مدت کوتاهی تمام توان پردازنده پر شده و scheduler نمیتواند به درستی فرایند‌ها را مدیریت کند، به همین علت دستور top یا kill اجرا نمیشوند، چون cpu اشغال است.

جدول فرایند‌ها (proccess table): ظرفیت این جدول محدود است و پس از مدتی کاملاً پر می‌شود و هیچ فرایند جدیدی نمیتوان ایجاد کرد.

I/O دیسک: با تمام شدن ram، سیستم به سراغ فضای swap دیسک رفته و سرعت سیستم بسیار کند شده و حتی موس و کیبورد هم پاسخ نمی‌دهند.

جدول فایل‌های باز (file descriptor table): از آنجا که هر فرایند تعدادی فایل باز میکند، این جدول پر شده و ادامه fork‌ها شکست می‌خورد.

علت بی‌تأثیر بودن Ctrl + C:

این دستور فقط به فرایند جاری (که فرایند والد است) ارسال شده و بر روی دیگر فرایندها تأثیر ندارد. فرایندهای فرزند در پس زمینه در حال اجرا اند و `ctrl + C` بر روی فرایندهای پس زمینه تأثیری ندارد. حتی دستور `kill` نیز کار نمی کند چون برای اجرای آم نیاز به اجرای فرایند جدید است اما جدول فرایند پر شده است.

۴. پیشگیری:

محدود کردن تعداد فرایندهای کاربر با `ulimit`، ایجاد محدودیت برای سرویس ها در `systemd`، جلوگیری از اجرای توابع ناشناس و جلوگیری از تعریف توابع با نام های عجیب مانند : ، محدود کردن استفاده از دستور `fork`

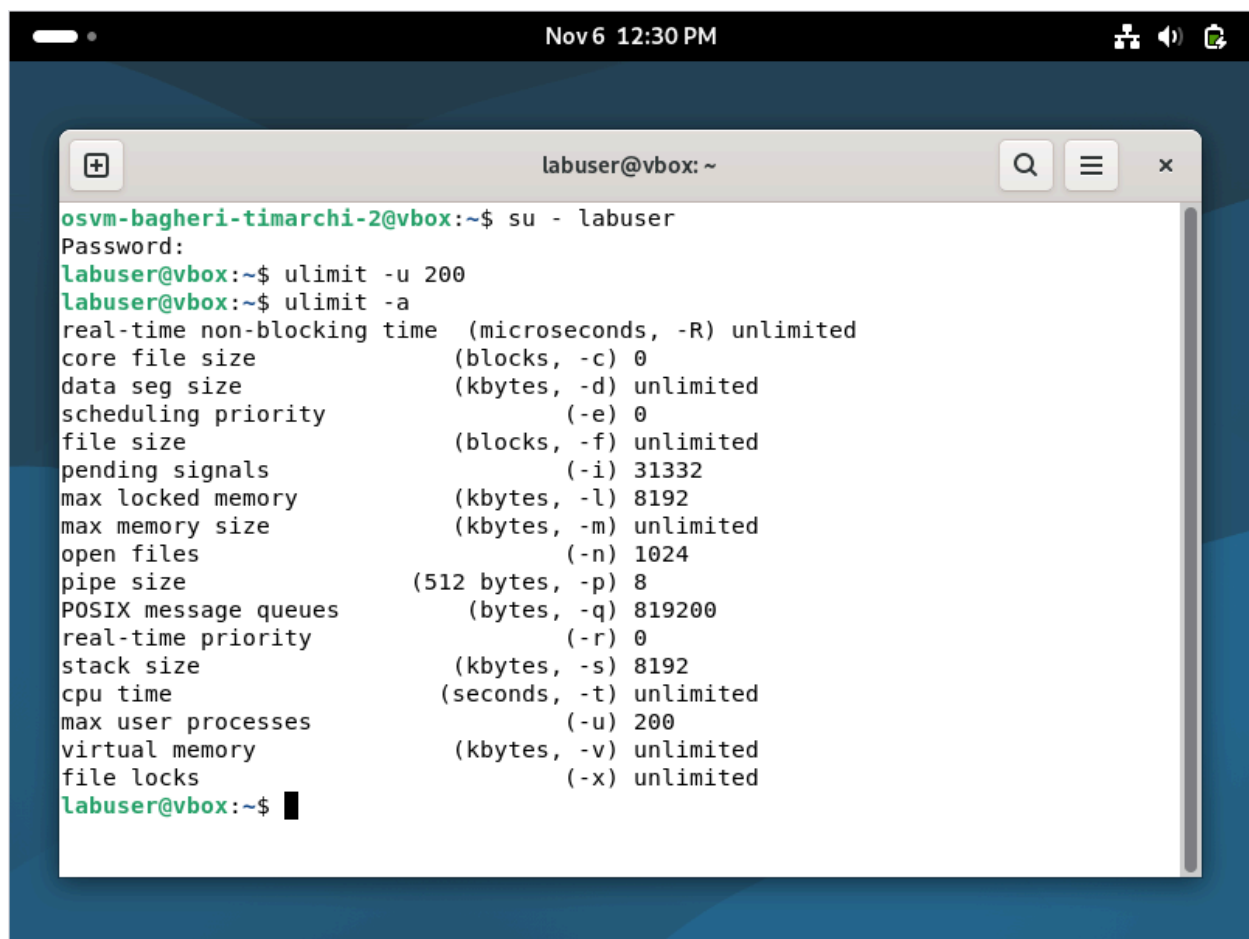
تشخیص: مانیتور کردن تعداد فرایندها (`nproc`)، قرار دادن آلارم با استفاده از `watch` یا `cron` یا ابراز های پیشرفته تر (آلارم وقتی `node_fork_rate > 100/sec`)

متوقف کردن در لحظات ابتدایی اجرا: کشتن فرایند والد، کشتن همه فرایندهای کاربر، کشتن همه فرایندهای کنسول، نهایتاً ری استارت فیزیکی

برای روشن کردن مجدد سیستم، از `recovery mode` استفاده شود

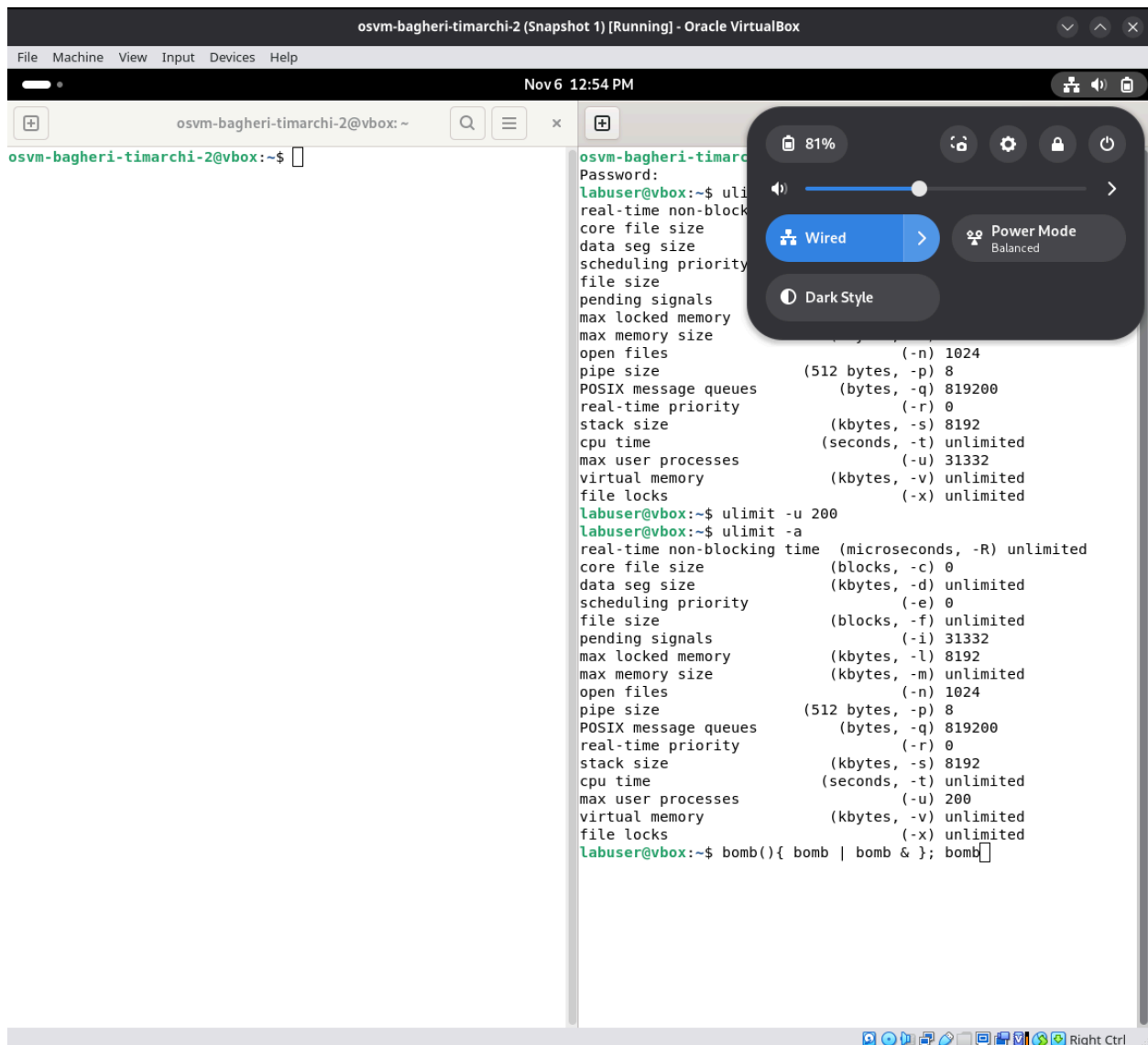
استفاده از داکر، کانتینر، `vm`، ایزوله سازی منابع

۵. ابتدا یک کاربر جدید ساخته و تعداد فرایندهای آن را محدود میکنیم:



```
Nov 6 12:30 PM
osvm-bagheri-timarchi-2@vbox:~$ su - labuser
Password:
labuser@vbox:~$ ulimit -u 200
labuser@vbox:~$ ulimit -a
real-time non-blocking time (microseconds, -R) unlimited
core file size (blocks, -c) 0
data seg size (kbytes, -d) unlimited
scheduling priority (-e) 0
file size (blocks, -f) unlimited
pending signals (-i) 31332
max locked memory (kbytes, -l) 8192
max memory size (kbytes, -m) unlimited
open files (-n) 1024
pipe size (512 bytes, -p) 8
POSIX message queues (bytes, -q) 819200
real-time priority (-r) 0
stack size (kbytes, -s) 8192
cpu time (seconds, -t) unlimited
max user processes (-u) 200
virtual memory (kbytes, -v) unlimited
file locks (-x) unlimited
labuser@vbox:~$
```

سپس fork bomb را اجرا میکنیم:



نمایش و تحلیل کارایی برنامه

۱. perf یک ابزار قدرتمند برای پروفایلینگ (Profile) و تحلیل عملکرد (Performance Analysis) در لینوکس است. این ابزار به ما کمک می‌کند تا بفهمیم:

- کدام توابع یا بخش‌های برنامه بیشترین زمان CPU را مصرف می‌کنند.
 - چند بار branch miss، cache miss یا context switch رخ داده است.
 - عملکرد سیستم یا برنامه در زمان اجرا چگونه بوده است.
۲. تمام این برنامه‌های اولیه و بهینه شده در فولدر این سوال موجود اند.

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace std;
```

```
double slowFunction() {
```

```
    double result = 0;
```

```
    for (long i = 0; i < 500000000; i++)
```

```
        result += sin(i) * cos(i/2.0);
```

```
    return result;
```

```
}
```

```
int main() {
```

```
cout << "Result: " << slowFunction() << endl;

return 0;

}
```

۳. به کمک دستورات زیر

```
g++ main.cpp -o main -O0
```

```
perf record ./main
```

```
perf report
```

ابتدا برنامه را کامپایل کرده، سپس اجرا کرده و از آن گزارش گرفتیم. که گزارش متنی از عملکرد برنامه بود. برای درک بهتر آن، در قسمت بعد آن را طور تصویری مشاهده می کنیم.

۴. FlameGraph ابزاری است برای نمایش بصری پروفایلینگ.

این ابزار از داده های perf استفاده کرده و آن را به صورت یک نمودار شعله ای نمایش می دهد که نشان می دهد: چه توابعی بیشتر اجرا می شوند. و زنجیره ی فراخوانی توابع به چه صورت است.

۵. با این دستورات خروجی های تصویری مورد نظر را ساختیم:

```
perf script > out.perf
```

```
git clone https://github.com/brendangregg/FlameGraph.git
```

```
cd FlameGraph
```

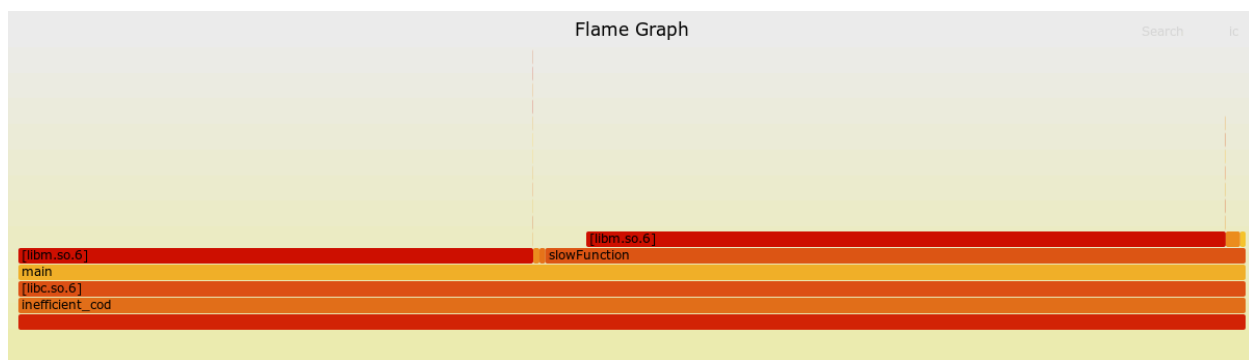
```
./stackcollapse-perf.pl ../out.perf > out.folded
```

```
./flamegraph.pl out.folded > perf_flame.svg
```

البته در حین نصب نیاز به تنظیم جزییات بیشتری بود، اما دستورات کلیدی را در بالا آوردم.

```
FlameGraph : bash — Konsole
New Tab Split View
Copy Paste Find...
bagheri-timarchi@debian:~/Desktop$ g++ -g -o inefficient_code inefficient_code.cpp -lm
bagheri-timarchi@debian:~/Desktop$ sudo perf record -F 99 -g ./inefficient_code
Result: 1.23969
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.455 MB perf.data (4720 samples) ]
bagheri-timarchi@debian:~/Desktop$ cd FlameGraph/
bagheri-timarchi@debian:~/Desktop/FlameGraph$ sudo perf script -i ../perf.data | ./stackcollapse-perf.pl > out.folded
Filtering for events of type: cpu_atom/cycles/P
bagheri-timarchi@debian:~/Desktop/FlameGraph$ ./flamegraph.pl out.folded > perf_flame.svg
bagheri-timarchi@debian:~/Desktop/FlameGraph$
```

تصاویر زیر نتایج حاصل برای کد بهینه نشده است. مجدداً یاد آور میشوم که تمام نتایج و کد ها در همین پوشه موجود اند:



۶. برای بهینه سازی، مهم ترین کاری که کردیم استفاده از موازی سازی بود به طوری که:

```
#include <iostream>

#include <cmath>

#include <numeric>

#include <omp.h> // Include OpenMP header

using namespace std;

double slowFunction() {

    double result = 0.0;

    long N = 500000000;

    // Use OpenMP to parallelize the loop and reduce the final results

    // The 'reduction(+:result)' clause safely combines the intermediate results

    // calculated by each thread into the final 'result' variable.

    #pragma omp parallel for reduction(+:result)

    for (long i = 0; i < N; i++) {

        // Use the double-precision math functions from <cmath>

        result += sin((double)i) * cos((double)i / 2.0);
```

```

    }

    return result;
}

int main() {

    // Optional: Measure execution time for comparison

    // double start_time = omp_get_wtime();

    cout << "Result: " << slowFunction() << endl;

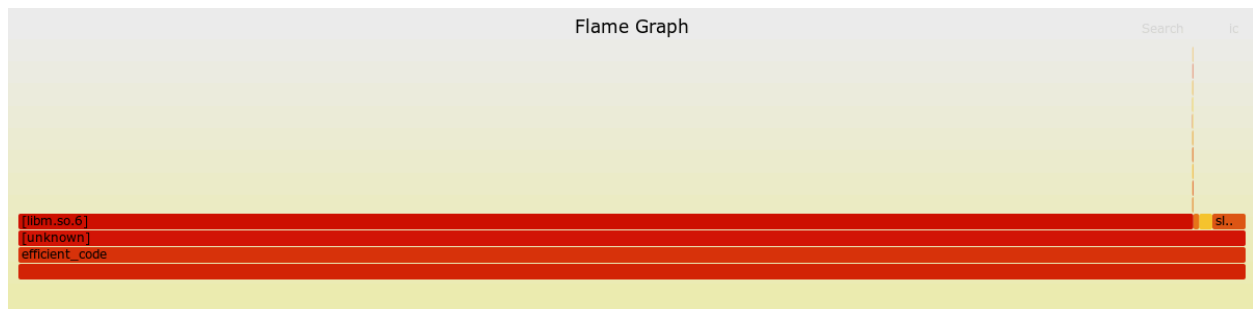
    // double end_time = omp_get_wtime();

    // cout << "Execution Time: " << (end_time - start_time) << " seconds" << endl;

    return 0;
}

```

۷. پس از بهینه سازی تصویر توابع به شکل زیر در آمد که با حالت اولیه تفاوت دارد:



۸. Tracy یک real-time profiler مدرن است که برای بازی ها و نرم افزارهای گرافیکی استفاده می شود.

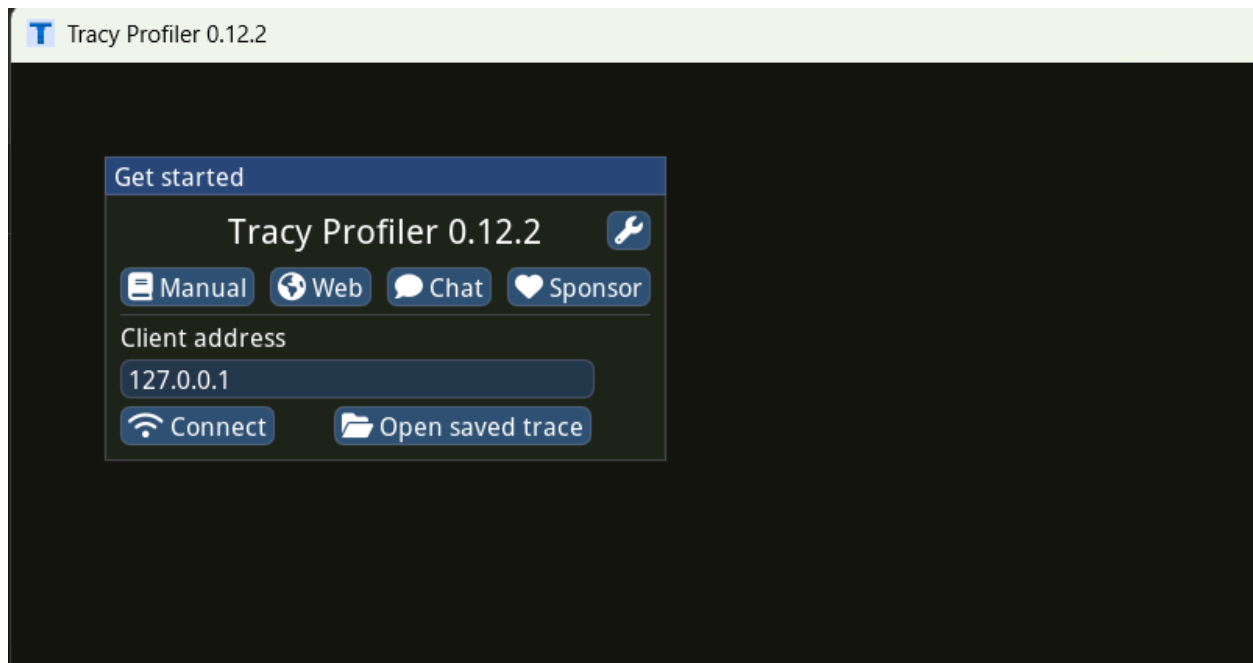
ویژگی ها:

- نمایش زنده ی مصرف CPU و GPU.

- مشاهده‌ی memory allocation و frame time در زمان اجرا.

- نیاز دارد کتابخانه‌ی Tracy را در کد خود include کنید و سرور Tracy را اجرا کنید.

۹. برای این کار بر روی ویندوز gui tracy را نصب کردیم:



کار کنند، مانند مورد زیر به آن ها خط های مورد نیاز را اضافه کردیم tracy برای اینکه برنامه ها با

```
#include <iostream>
```

```
#include <cmath>
```

```
// 1. Include the Tracy header
```

```
#include "Tracy.hpp"
```

```
using namespace std;
```

```
double slowFunction() {
```

```
    // 2. Add the profiling macro at the top of the function
```

```
    ZoneScoped;
```

```

double result = 0;
for (long i = 0; i < 500000000; i++)
    result += sin(i) * cos(i/2.0);
return result;
}

int main() {
    // Optional: Profile the main function as well
    ZoneScoped;

    cout << "Result: " << slowFunction() << endl;

    // Optional: Call FrameMark at the end of the loop/frame
    FrameMark;

    return 0;
}

```